



Taylor & Francis
Taylor & Francis Group



A Systematic Approach to Automatic Edit and Imputation

Author(s): I. P. Fellegi and D. Holt

Source: *Journal of the American Statistical Association*, Vol. 71, No. 353 (Mar., 1976), pp. 17-35

Published by: Taylor & Francis on behalf of the American Statistical Association

Stable URL: <https://www.jstor.org/stable/2285726>

Accessed: 13-09-2026 15:33 UTC

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at <https://about.jstor.org/terms>



JSTOR

Taylor & Francis, American Statistical Association are collaborating with JSTOR to digitize, preserve and extend access to *Journal of the American Statistical Association*

A Systematic Approach to Automatic Edit and Imputation

I. P. FELLEGI and D. HOLT*

This article is concerned with the automatic editing and imputation of survey data using the following criteria.

1. The data in each record should be made to satisfy all edits by changing the fewest possible items of data (fields);
2. As far as possible the frequency structure of the data file should be maintained;
3. Imputation rules should be derived from the corresponding edit rules without explicit specification.

A set of procedures is developed for identifying the fewest number of fields which may be changed to make the resulting data satisfy all edits. Several methods are proposed for determining the specific values to be imputed into each field.

1. INTRODUCTION

The last stage of the processing of survey data, just prior to tabulations and data retrieval, usually involves some form of editing. By editing we mean

- A. the checking of each field of every survey record (the recorded answer to every question on the questionnaire) to ascertain whether it contains a valid entry; and
- B. the checking of entries in certain predetermined combinations of fields to ascertain whether the entries are consistent with one another.

Checking Type A may involve determining whether the entry of any field is an invalid blank (for some fields blank entries may be valid—e.g., the reported income of minors—but we mean here invalid blanks), and whether the recorded entries are among a set of valid codes for the field. Examples are: age should not be blank or negative; marital status should not be blank; number of children should be less than 20.

While edits of Type A are immediate consequences of the questionnaire and code structure, edits of Type B are usually specified on the basis of extensive knowledge of the subject matter of the survey. Conceptually, edits of Type B specify in one form or another sets of values for specified combinations of fields which are jointly unacceptable (or, equivalently, sets of values which are jointly acceptable). Examples are: if age is less than 15 years, then marital status should be single; the acreage reported to be under different specified crops should be

equal to total reported acreage under cultivation; the total value of production reported in a given industry divided by total manhours worked should be between two predetermined constants.

This article essentially deals with computer edits. When a record fails some of the edits, we have, theoretically, five options:

1. Check the original questionnaires in the hope that the original questionnaire is correct and the edit failures are caused by coding error or error introduced during the conversion of data to machine-readable form;
2. Contact the original respondent to obtain the correct response (or verify that the reported response was correct in its original form);
3. Have clerical staff "correct" the questionnaire using certain rules which would remove the inconsistencies;
4. Use the computer to "correct" the questionnaire, also using certain rules which would remove the inconsistencies;
5. Drop all records which fail any of the edits or at least omit them from analyses using fields involved in failed edits.

Option 5 would involve an implicit assumption that the statistical inferences are unaffected by such deletions. This is equivalent to assuming that the deleted records have the same distribution as the satisfactory records. If such an assumption must be made it would seem much more sensible to make it through imputation techniques. At any rate, when population totals have to be estimated, *some* form of correction would still be necessary: if the correction is not made through explicit imputation, it would have to be made through some form of weighting. However, implicit in weighting is the imputation of the appropriate mean to all fields of all records which are involved in failed edits. We believe that a better imputation results if we make use of the valid parts of questionnaires and impute for as few fields as possible.

Corrections of Type 1 and 2, particularly 2, are certainly worthwhile if feasible: one should, whenever possible, avoid "manufacturing" data instead of collecting it. Often, however, it is not possible to recontact the respondents, mostly due to limitations of time and money. In these cases, 1, 3, and 4 are left open to us. However, the incidence of cases where 1 is fruitful can and probably should be minimized by a rigorous application of quality controls to the coding and data conversion operations.

*I.P. Fellegi is assistant chief statistician of Canada, Statistical Services Field, Statistics Canada, Ottawa, Ontario K1A 0T6. D. Holt is lecturer, Department of Econometrics and Social Statistics, University of Southampton, Southampton, Hants, SO9 5NH, England. The authors would like to acknowledge many useful discussions with their colleagues at Statistics Canada and also with T.M.F. Smith while on leave of absence from the University of Southampton. Particular thanks are due R. Graves and M. Podehl for their insightful contributions and leadership in implementing the generalized system based on the theories and methods of this article.

At any rate, whether we use 1 and 2, we usually reach a point where 3 or 4 has to be applied. Of these alternatives, we have a strong preference for 4: if the data are to be "corrected" using predetermined rules, it is usually much more preferable, although not necessarily more economical, to let the computer apply these rules rather than clerks. Contrary to clerks, the computer applies the rules consistently and fast; moreover, if editing is done by a computer but correction by clerks, usually the data have to be edited again to insure that all inconsistencies have been removed. This may lead to a long series of computer edits alternating with clerical corrections and the interface between the two operations involves the usually relatively slow process of input and output, as well as potentially complex logistic and control operations.

The major disadvantage of a procedure involving the computer for both editing *and* correction is the potential programming complexity and the relative rigidity of the computer programs, i.e., the complexity of the programs is usually such that changes are time-consuming, expensive and error-prone. Edit and correction rules are often more or less independently specified with no full assurance that the corrections will render the data consistent with respect to the edits. Also, in the absence of some overall philosophy of correction and in the light of the very complex rules often used, the net impact of the corrections on the data is unforeseeable and, what is possibly even worse, not readily capable of a systematic post-implementation evaluation.

This article proposes an approach which potentially overcomes (or at least ameliorates) these problems. With respect to complexity, the approach presented simplifies the programming task by identifying a uniform and particularly simple form for all edits, which also assists in overcoming the potential rigidity of computer programs. In fact, the "rigidity" usually arises because the edits are frequently specified in the form of logically complex and interrelated chains or networks. The article breaks these down into a series of *simple* and *unrelated* edit rules of a common form; once this form of edits is implemented it is relatively easy to add additional edit rules or remove some of the edits already specified. This ease of modifying the edits should facilitate carrying out a set of initial edits in exploratory fashion (possibly on a sample of records) and their modification, as desirable, prior to data correction.

In effect, the approach lends itself to the implementation of a generalized edit and data correction system driven by subject-matter specification, consisting of specifying inconsistent combinations of codes. The simplicity with which edit specifications can be altered without the need to reprogram provides the most important payoff of the approach: it facilitates experimentation with alternative edit specifications and permits evaluation of their impact on the "corrected" data. Should this impact be found unacceptable, the entire edit system can be respecified with ease and rerun without delay.

Data correction is often the most complex of the survey data processing operations. This article presents an approach which, optionally, altogether eliminates the necessity for a separate set of specifications for data corrections: the needed corrections are automatically deduced from the edits themselves. On one hand, this should simplify specification of correction routines and, on the other hand, insure that the corrections are always consistent with the edits. The consistency of corrections with edits becomes a particularly important issue if we want the flexibility of changing the initially specified edits. A fuller evaluation of the benefits offered by this approach is presented in Section 6.

In an earlier paper, Freund and Hartley [1] proposed a least squares type approach to data correction in the case of quantitative variables with arithmetic edits. The approach assumed a metric underlying the data. However, for qualitative data (i.e., coded data such as sex, occupation, etc.) there is often no basis for assuming a metric, and such an approach to data correction (imputation) is not realistic. In fact, the central concern of this article is with qualitative data, although some of the theoretical results also apply to quantitative data. These will be pointed out specifically. A subsequent paper by one of the authors deals with quantitative data specifically.

There are three criteria for imputation of qualitative data which we have attempted to meet, the first of which is overridingly important in the rest of this article.¹

1. The data in each record should be made to satisfy all edits by changing the fewest possible items of data (fields). This we believe to be in agreement with the idea of keeping the maximum amount of original data unchanged, subject to the constraints of the edits, and so manufacturing as little data as possible. At the same time, if errors are comparatively rare, it seems more likely that we will identify the truly erroneous fields. This criterion appears to be reasonable, particularly for qualitative (coded) data, since it provides the only feasible measure of the extent of changes due to imputations.
2. It should not be necessary to specify imputation rules; they should derive automatically from the edit rules. This would insure that imputed values will not continue to fail edits, simplify the task of specifying edits and imputations, simplify their computer implementation, facilitate the implementation of subsequent changes to specifications, and generally lead to a more controlled operation.
3. When imputation takes place, it is desirable to maintain, as far as possible, the marginal and even preferably the joint frequency distributions of the variables, as reflected by the "correct" records, i.e., those which pass the edits

The idea of Criterion 3 can be explained in terms of an example. Suppose that in a survey we collect data about age, sex, occupation and labor-force status. If, say, in a given record, labor-force status is identified as having been reported in error, usually we know no more about the particular record than that it represents an individual

¹ By imputation for a given record we mean changing the values in some of its fields to possible alternatives with a view to insuring that the resultant data record satisfies all edits. To the extent that an originally recorded value may be an invalid blank, the term encompasses data correction necessitated by partial nonresponse or by prior clerical editing.

belonging to that subpopulation which has identical age, sex and occupation to that shown on the given record. In the absence of other information, the logical inference about labor-force status would be the average labor-force status of this subpopulation as measured by the error-free records; since the average of a set of codes does not make sense, we replace the average by a code value randomly selected from this subpopulation. Even in the case of quantitative variables, where the average is a sensible quantity, a suitably chosen value from the subpopulation might still be preferable, since imputing the average repeatedly would distort the distributions.

In concluding this introduction, we emphasize that the motivation for this article is *not* to make editing and imputation so simple and routine as to tempt survey takers and processors to replace direct observation and measurement by computer imputations. The approach to editing is predicated on the assumption that one wants to alter the original observations to the least possible extent. We provide a systematic approach and tools to achieve this. While the amount of imputation is thus minimized, it may still be unacceptably high in particular surveys, depending on the quality of data collection and conversion, relative to the particular requirements of statistical inference for which the survey was taken. Thus, while the question of the impact of automatic editing and imputation on the subsequent statistical inference is outside the scope of this article, we strongly recommend that this impact be studied in the context of the particular applications.

2. THE NORMAL FORM OF EDITS

At the beginning, let us restrict ourselves to records containing only qualitative (coded) data, i.e., data which are not subject to a meaningful metric. Edits involving such data will be referred to as *logical edits*, as opposed to *quantitative arithmetic edits*, which are meaningful only in the presence of a metric.

Edits of qualitative (coded) data, as expressed by experts with a knowledge of the subject matter of the survey, are usually conveyed in the form of narratives, flowcharts, or decision tables. Whatever the medium, however, an edit expresses the judgment of some experts that certain combinations of values or code values in different fields (corresponding to the different questions on the questionnaire) are unacceptable.

Let each editable record contain N fields. Denote by A_i the set of possible values or code values which may be recorded in Field i , the number of such values being n_i (not necessarily finite).

The concept that a particular record (questionnaire), say, $\mathbf{a}$, has in Field i a code belonging to some subset of A_i , say, A_i^0 , can be concisely denoted by

$$\mathbf{a} \in A_i^0. \quad (2.1)$$

Mathematically, this notation is not entirely rigorous because $\mathbf{a}$, being a vector of N components, is not a

member of the set of scalars A_i^0 . What we really understand by (2.1) is that $\mathbf{a}$ is a member of the Cartesian product $A_1 \times A_2 \times \cdots \times A_{i-1} \times A_i^0 \times A_{i+1} \times \cdots \times A_N$. However, we will simplify the notation by using A_i^0 instead of the corresponding Cartesian product—the context always clarifying whether A_i^0 is actually a subset of A_i or a subset of the total code space $A_i \times A_2 \times \cdots \times A_N$.

Now the concept of an edit restriction can be made more precise. The combination of code values in different fields which are unacceptable is a subset of the code space and can, in general, be expressed as

$$f(A_1^0, A_2^0, \dots, A_N^0), \quad (2.2)$$

where A_i^0 are subsets of the code space and the function f connects these subsets through the $\cap$ (set intersection) and $\cup$ (set union) operations. The function f thus defines a subset of the code space and becomes an edit if, through some prior knowledge, it is declared a set of unacceptable code combinations, in the sense that a record $\mathbf{a}$ is said to *fail* the edit specified by f whenever

$$\mathbf{a} \in f(A_1^0, A_2^0, \dots, A_N^0). \quad (2.3)$$

A repeated application to f of the “distributive law” connecting the set union and intersection operations will transform the right side of (2.3) to a form consisting of the union of intersections of sets given by

$$\begin{aligned} f(A_1^0, A_2^0, \dots, A_N^0) &= (A_{i_1}^1 \cap A_{i_2}^1 \cdots \cap A_{i_{m_1}}^1) \\ &\cup (A_{j_1}^2 \cap A_{j_2}^2 \cdots \cap A_{j_{m_2}}^2) \\ &\cup \cdots \cup (A_{k_1}^r \cap A_{k_2}^r \cdots \cap A_{k_{m_r}}^r). \end{aligned} \quad (2.4)$$

Now it is easy to show that $\mathbf{a} \in f$ will occur if and only if $\mathbf{a}$ is a member of any one (or more) of the sets defined by the brackets on the right side of (2.4). So the edit defined by (2.3) can be broken down into a *series* of edits of the form

$$\mathbf{a} \in \bigcap_{i \in s} A_i^*, \quad (2.5)$$

where s is a suitably defined index set on the fields of the questionnaire, A_i^* are subsets of the code space of the form given by (2.1) and (2.5) defines a subset of the code space such that any record in it fails an edit.

What (2.5) states is perhaps intuitively obvious: any edit, whatever its original form, can be broken down into a series of statements of the form “a specified combination of code values is not permissible.”

Several points should be noted.

1. Formula (2.5) indicates that instead of testing each record as to whether it falls into the kinds of edit failure sets shown on the left side of (2.4), we can test them with respect to the simpler sets on the right side of (2.5). We will write

$$\bigcap_{i \in s} A_i^* = F \quad (2.6)$$

to indicate that the set on the left side is such that any record in it would be an edit failure. Moreover, clearly,

(2.6) is not altered substantively if we write instead

$$\bigcap_{i=1}^N A_i^* = F, \quad (2.7)$$

where for all fields $i \in s$ we put $A_i^* = A_i$ (the set of all possible code values for Field i). We say that Field i enters an edit of the form (2.7) *explicitly* if A_i^* is a proper subset of the set of codes for Field i .

2. Formulas (2.6) or (2.7) will be referred to as the *normal form of edits*. What we have shown is that any complex edit statement can be broken down into a *series* of edits, each having the normal form. Since the derivation of (2.6) involved only the repeated application of the distributive law connecting the $\cap$ and $\cup$ operators, it provides an operationally feasible procedure of recasting edit rules into a series of edits in the normal form. We will assume throughout this article that, unless otherwise stated, all edit rules are provided in the normal form. The set of such edit rules in the normal form, as specified by the subject matter experts, will be referred to as *explicit edits* (to distinguish them from logically implied edits—a concept to be introduced later).

3. Hereafter it will be assumed that the *a priori* specified edits (usually provided by experts of the subject matter of the survey) are in the normal form (2.6) or (2.7). However, before proceeding, it is relevant to ask: How much extra work do we shift on the subject-matter experts by requiring them to adhere to formal syntax provided by the normal form? The language of edit specifications which they are used to contains, essentially, two types of statements.

- a. Simple validation edits, stating that the permissible code values for Field i are given by the set A_i , any other value being an error (response error, data conversion error, etc.). This can be converted into the normal form very easily and automatically. For each field the set A_i can be expanded to include a single code, say, c_i , which is to represent all the invalid codes of Field i . During the loading of the unedited data into a data base (or some other equivalent operation), all invalid codes (including invalid blanks) in Field i are replaced by the code c_i . Now a series of edits of the normal form

$$\{c_i\} = F \quad i = 1, 2, \dots, N$$

is equivalent to other forms of specifying the simple validation edits.

- b. More complex consistency edits involving a finite set of codes are typically of the form: "Whenever $a \in f(A_1', A_2', \dots, A_N')$, then it should follow that $a \in g(A_1'', A_2'', \dots, A_N'')$," i.e., whenever a record has certain combinations of code values in some fields, it should have some other combinations of code values in some other fields. Since g has the same structure as f (both being constructed from unions or intersections of code sets), we can take the complement of the set represented by g (which will also formally have the same structure as f); then the edit statement just given is logically equivalent to: "Whenever $a \in f \cap \bar{g}$, then the record fails the edit." This latter statement, however, is formally equivalent to (2.3), so it can also be converted into the normal form (2.6). Hence, whether the originally specified edits are given in the form which define edit failures explicitly or in a form which

describe conditions that must be satisfied, the edit can be converted into a series of edits in the normal form, each specifying conditions of edit failure.

Since the conversion of consistency edits into a series of edits in the normal form is an operation which can be completely specified in terms of an algorithm whose steps consist only of the elementary operations of Boolean algebra (taking complements, applying the distributive laws, etc.), a computer algorithm can easily be written to accomplish this, thereby permitting subject-matter experts to specify edits in the language they are used to. Despite the simplicity of such an algorithm, we decided not to implement it, but rather ask our subject-matter experts to adhere to the extremely simple syntax provided by the normal form. The advantages of doing so will become clear in Section 6. Suffice to say here, that the form of edit specification provided by (2.5) was, precisely due to its simplicity, fully embraced by subject-matter experts who were involved in the first application of the system based on the methodology of this article.

4. As has already been emphasized, we have to draw a sharp distinction between *logical edits* (i.e., edits involving a questionnaire, each field of which contains a finite number of codes) and *quantitative edits* (i.e., edits involving a questionnaire, each field of which is measured on a continuous scale). The normal form of edits is both a natural and operationally feasible way of identifying edit conflicts of the logical kind (i.e., that certain code combinations are declared edit failures). Arithmetic edits, as shown shortly, can be cast into the normal form, but this is not their natural form (i.e., a series of equalities or inequalities). However, since both logical and arithmetic edits *can* be cast in the normal form, the theoretical results of Section 3 (which depend on the *possibility* of casting edits in the normal form) apply to both logical and arithmetic edits. The implementation strategy of Sections 4 and 5, which is mostly based on the assumption that the edits are actually stated in the normal form, is likely to be practicable as stated only for logical edits. Possible implementation strategies for arithmetic edits are specifically indicated whenever relevant, but they will be taken up in more detail in a forthcoming paper by one of the authors.

We will indicate in the following how linear arithmetic edits can be expressed in the normal form.

We postulate (although this is not entirely necessary) for the remainder of this article that all arithmetic expressions involved in the edit specifications are linear. Such expressions may involve equalities or inequalities and would take the following form. Whenever

$$0 \leq M(a_1, a_2, a_3, \dots, a_N),$$

then the record fails this edit; where $a_1, a_2, a_3, \dots, a_N$ are variables corresponding to the fields of the questionnaire and can assume values in the sets $A_1, A_2, \dots, A_N$; and where the function M is a linear expression. With-

out loss of generality, suppose that a_1 has a nonzero coefficient.

Then one can rewrite the preceding expression as

$$a_i \leq L(a_2, a_3, \dots, a_N),$$

where L is also a linear function. Finally, this inequality can be expressed in a normal form as a set of edits

$$A_1^0 \cap A_2^0 \cap A_3^0 \cap \dots \cap A_N^0 = F,$$

where

$$A_1^0 = \{a_1: a_1 \leq L(a_2, a_3, \dots, a_N)\}$$

$$A_2^0 = \{a_2\}, \quad A_3^0 = \{a_3\}, \dots, \quad A_N^0 = \{a_N\},$$

and there is one such set of edits corresponding to each possible choice of the values $a_2, a_3, \dots, a_N$ such that $a_2 \in A_2, a_3 \in A_3, \dots, a_N \in A_N$.

To illustrate, suppose that in a survey of farms, three fields on a questionnaire relate, respectively, to number of cultivated acres, unimproved acres, and total acres of land. The corresponding variables are a_1, a_2 , and a_3 , the set of possible values of the three variables being A_1, A_2 , and A_3 (each presumably the set of nonnegative numbers). An edit may be

$$a_1 + a_2 \neq a_3,$$

i.e., that a record satisfying this relation is an edit failure. This can be transformed into a set of edits in the normal form simply as

$$\{a_3: a_3 \neq a_1 + a_2\} \cap \{a_1\} \cap \{a_2\} = F, \\ \text{for all } a_1 \geq 0 \text{ and } a_2 \geq 0;$$

i.e., for any a_1 and a_2 a combination of the values a_1, a_2 , and any a_3 which is not equal to the sum of a_1 and a_2 is an edit failure.

5. Often the set of possible values of all fields is finite. Such is the case, for example, if all the characteristics on the questionnaire involve qualitative codes (e.g., sex, industry, labor-force status). Even if some of the characteristics involve a conceptually infinite or a very large number of possible code values (such as age, income, etc.), the edits may only distinguish explicitly some finite number of ranges. For example, if income enters only two edits distinguishing between the ranges 5,000–10,000 and 7,000–12,000, respectively, then the totality of all edits requires distinguishing only between the five ranges 0–4,999, 5,000–6,999, 7,000–10,000, 10,001–12,000, 12,000+, and we may consider for purposes of editing the field “income” as assuming only five possible code values. In such cases quantitative fields can be treated for purposes of editing and imputation as if they were qualitative (coded) fields.

When the set of possible values of all fields is finite, a simple possible implementation of an edit system is provided in Section 5.

6. We accept a convention that whenever one of the fields of a record contains an invalid entry (i.e., one which is not among the set of possible values), we con-

sider the record as failing all edits which that field enters explicitly. The code value “blank” may or may not be among the set of possible (i.e., permissible) values of a field; e.g., blank in the income field might be a possible value in a general population survey, whereas blank in the age field would not be a possible value.

Example: The following example (clearly not realistic) illustrates the procedure of converting edits into the normal form.

Suppose in a demographic survey one of the edits specified by subject-matter experts is: “If a person’s age is ≤ 15 years or he (she) is an elementary school student, then relationship to head of household should not be head and marital status should be single.”

This edit can be converted into the normal form in a series of steps:

$[(\text{Age} \leq 15) \text{ or } (\text{Elementary School})]$ implies $[(\text{not Head}) \text{ and } (\text{Single})]$

$[(\text{Age} \leq 15) \text{ or } (\text{Elementary School})]$ and not $[(\text{not Head}) \text{ and } (\text{Single})] = \text{Failure}$

$[(\text{Age} \leq 15) \text{ or } (\text{Elementary School})]$ and $[(\text{Head}) \text{ or } (\text{not Single})] = \text{Failure}$

$(\text{Age} \leq 15) \text{ and } (\text{Head}) = \text{Failure}$

$(\text{Age} \leq 15) \text{ and } (\text{not Single}) = \text{Failure}$

$(\text{Elementary School}) \text{ and } (\text{Head}) = \text{Failure}$

$(\text{Elementary School}) \text{ and } (\text{not Single}) = \text{Failure}$

The last four statements together are equivalent to the originally specified edit. They are in the normal form.

3. THE COMPLETE SET OF EDITS

A record which passes *all* the stated edits is said to be a “clean” record, not in need of any “correction.” Conversely, a record which fails *any* of the edits is in need of some corrections. Unfortunately, generally one knows only which edits are failed, but not which fields are causing the edit failures. In this section we will try to build a bridge leading to an inference from the knowledge of the edits which failed to the identification of the fields which need to be changed to remove the edit failures. The main idea can be intuitively summarized as follows (a precise treatment is given subsequently). Suppose that a record fails some of the specified edits. For the sake of simplicity, suppose also that there is a *single* field, say, Field i , which enters *all* the failed edits. Then it should be possible to find a value for that field which will convert all the failed edits to satisfied ones (and not convert any satisfied edits to failed ones). In fact, if this could not be done, then this would mean that in the presence of the current values in the other fields, all the possible values of Field i result in some edit failures; in this case, so to speak, the value in Field i is irrelevant; the *other* fields are the cause of at least some of the edit failures. So, one might feel, there must be at least one failed edit which involves the *other* fields but not Field i . This would *appear* to contradict our initial assumption, namely that Field i enters all the failed edits. Indeed, this would be an *actual* contradiction if all the edits were explicitly stated.

As it is, often the initially specified edits *imply* logically some other edits. However, if we could be sure that *all* edits (explicit and implied) are explicitly known, under the simple assumptions stated it should be possible to convert all failed to satisfied ones by changing the value in a single field. Thus, while the initially specified edits are all that are necessary to identify the records which pass or fail the edits, the edits implied by them logically must also be considered if one wishes to investigate systematically the field(s) which must be changed to “correct” the record (change it in such a fashion that the changed record passes all the edits).

Some examples might further clarify the notion.

Example 1: Suppose that a questionnaire contains three fields.

Field	Possible codes
Age	0-14, 15+
Marital Status	Single, Married, Divorced, Widowed, Separated
Relationship to Head of Household	Head, Spouse of Head, Other

Suppose there are two edits,

- I $(\text{Age} = 0-14) \cap (\text{Mar. Stat.} = \text{Ever married}) = F$
- II $(\text{Mar. Stat.} = \text{Not Now Married}) \cap (\text{Rel. to Head} = \text{Spouse}) = F,$

where Ever Married stands for the subset of marital status codes {Married, Divorced, Widowed, Separated} and Not Now Married stands for the subset of codes {Single, Divorced, Widowed, Separated}.

Suppose that a record has the values Age = 0-14, Marital Status = Married, Relationship to Head of Household = Spouse. This record fails Edit I, passes Edit II. In an attempt to try to correct this record through imputation, we may consider changing the field Marital Status. It is easy to verify that, leaving Age and Relationship to Head of Household unchanged, all possible Marital Status codes would result in a record which would fail one or the other of the two edits. One would suspect, therefore, that there must be a hidden conflict between the current values of Age and Relationship to Head of Household—irrespective of Marital Status. In fact, in this simple example, it is intuitively clear that there is a conflict between the age being between zero and 14 years and relationship to head of household being spouse. The existence of such a conflict, as one which is implied logically by the two stated edits, can be formally established as follows (a more rigorous method is provided by the Lemma, stated later in this section).

Edits I and II can be restated (using the arrow $\rightarrow$ to mean “implies”) as

- I $(\text{Age} = 0-14) \rightarrow (\text{Marital Status} = \text{Single})$
- II $(\text{Marital Status} = \text{Not Now Married}) \rightarrow (\text{Rel. to Head} = \text{Not Spouse}).$

Since it is obviously true that

$$(\text{Marital Status} = \text{Single}) \rightarrow (\text{Marital Status} = \text{Not Now Married}),$$

we obtain, combining I and II,

$$(\text{Age} = 1-14) \rightarrow (\text{Rel. to Head} = \text{Not Spouse})$$

Finally, the preceding statement can be recast, equivalently, as

$$\text{III } (\text{Age} = 0-14) \cap (\text{Rel. to Head} = \text{Spouse}) = F$$

Thus Edit III is logically implied by Edits I and II.

It can be shown, using the methods provided later in this section, that no new edits can logically be derived from these three edits. A set of edits having this property will be called a *complete set of edits*.

We can now illustrate the main result of this section as follows. The current record, as just described, fails Edits I and III. In attempting to correct this record, we will select fields which, between them, explicitly figure in all the *failed edits*. We say that such fields “cover off” the failed edits. Now in this instance, Edits I and III are covered off either by the single field Age, or by any two of the three fields, or by the three fields together. According to the main theorem of this section, any combination of fields which covers off all the failed edits can be changed so that the resulting record will pass all edits. In this instance we can change Age to 15+ (leaving the other two fields unchanged) and this will result in a conflict-free record; alternatively, we could appropriately change any two fields (leaving the third field unchanged) to obtain a conflict-free record. In such a situation one can argue that the combined evidence presented by all the fields together seems to point to the single field of Age being in error, rather than some combination of two fields being in error.

The point to note in this example is the importance of identifying, together with the initial edits (Edits I and II), the logically implied edit (Edit III). Edits I and II only suffice to identify that the current record is subject to a conflict. However, it is only after having identified the logically implied Edit III that we are in a position to determine systematically the field(s) which have to be changed to remove all inconsistencies.

Example 2: Suppose that a questionnaire has four quantitative fields, the information recorded in these fields being denoted, respectively, by the variables *a*, *b*, *c*, and *d*. Suppose that there are two arithmetic edits connecting these variables, each indicating a condition of edit failure (it is a uniform convention adopted throughout that we specify conditions of edit failure, rather than the opposite conditions of edit consistency):

- I $a + c + d \leq b$ or, equivalently, $a - b + c + d \leq 0$
- II $2b \leq a + 3c$ or, equivalently, $-a + 2b - 3c \leq 0.$

Let the current record contain the values: *a* = 3, *b* = 4, *c* = 6, *d* = 1. It is easy to see that the first edit

is passed; the second is failed. It is not immediately clear, even in this simple example, which fields (variables) need to be changed to satisfy the edits. However, if we write the logically implied edits, the situation clarifies immediately. It is easy to verify that the following three inequalities are all implied by the preceding two (they are linear combinations with positive weights of those two inequalities):

$$\text{III } b - 2c + d \leq 0$$

$$\text{IV } a - c + 2d \leq 0$$

$$\text{V } 2a - b + 3d \leq 0.$$

Now, one can verify that Edits II, III, and IV fail; I and V are satisfied. Looking for variables which, between them, enter all the failed edits, it is immediately obvious that variable c enters all the failed edits, or alternatively, any pair of variables also enter (between them) all the failed edits. Thus, according to the results of this section, one can remove all inconsistencies by choosing a suitable value for c (leaving the values of the other three variables unchanged); alternatively, one would have to change the values of at least two variables. Thus, the only single variable which can be suitably changed is c . In effect, changing c to any value between zero and $5/3$ will result in a record which satisfies all the edits, e.g., $c = 1$ would be a suitable imputation.

Under more general assumptions we will identify a set of minimal number of fields which, when changed, will result in satisfying all the edits. But first we need a general method enabling us to write all the edits logically implied by the explicitly stated edits.

We will first state and prove a Lemma which provides a method of deriving implied edits from a set of edits. It will be shown later (Theorem 2) that *all* implied edits can be derived by repeated applications of the Lemma.

Lemma: If e_r are edits for all $r \in s$ where s is any index set,

$$e_r: \bigcap_{j=1}^N A_j^r = F \quad \text{for all } r \in s.$$

Then, for an arbitrary choice of i ($1 \leq i \leq N$), the expression

$$e^*: \bigcap_{j=1}^N A_j^* = F \quad (3.1)$$

is an implied edit, provided that none of the sets A_j^* is empty, where

$$A_j^* = \bigcap_{r \in s} A_j^r \quad j = 1, \dots, N; j \neq i$$

$$A_i^* = \bigcup_{r \in s} A_i^r.$$

The proof is presented in the Appendix.

It is relevant to emphasize that the edits $e_r (r \in s)$ in the statement of the Lemma may represent *any* subset of the set of edits. For example, if we start with, say, ten edits, we may try to derive an implied edit from

any two, three, four, etc., of them. Also, implied edits, once derived, can participate in the derivation of further implied edits. The edits from which a particular new implied edit is derived are called "contributing edits."

It will be shown subsequently (Theorem 2) that *all* implied edits can be generated through a repeated application of this Lemma. For reasons of presentation, we postpone the statement of Theorem 2 to the end of this section.

To illustrate the Lemma, we return to Example 1. There are two edits there; e_1 and e_2 are, respectively, Edits I and II of that example. These edits can be rewritten as

$$e_1: (\text{Age} = 0-14) \cap (\text{Rel. to Head} = \text{Any code}) \cap (\text{Mar. Stat.} = \text{Ever Married}) = F$$

$$e_2: (\text{Age} = \text{Any code}) \cap (\text{Rel. to Head} = \text{Spouse}) \cap (\text{Mar. Stat.} = \text{Not Now Married}) = F.$$

Letting Marital Status play the role of Field i of the Lemma (the generating field), we obtain

$$A_i^* = (\text{Mar. Stat.} = \text{Ever Married}) \cup (\text{Mar. Stat.} = \text{Not Now Married}) \\ = (\text{Mar. Stat.} = \text{Any code}).$$

While, for the other two fields,

$$A_1^* = (\text{Age} = 0-14) \cap (\text{Age} = \text{Any code}) \\ = (\text{Age} = 0-14)$$

$$A_2^* = (\text{Rel. to Head} = \text{Any code}) \cap (\text{Rel. to Head} = \text{Spouse}) \\ = (\text{Rel. to Head} = \text{Spouse}).$$

Thus we obtain an implied edit,

$$e^*: (\text{Age} = 0-14) \cap (\text{Rel. to Head} = \text{Spouse}) \cap (\text{Mar. Stat.} = \text{Any code}) = F.$$

This is the same implied edit derived earlier using a more heuristic approach.

If all the sets A_j^r are proper subsets of A_j , the complete set of code values for Field j , but

$$A_i^* = A_i,$$

then the implied edit (3.1) is said to be an *essentially new edit*. In fact, in this case (3.1) does not involve all the fields explicitly involved in the e_r (Field i being explicitly involved in the Edit e_r , but not in (3.1)). Field i is referred to as the *generating field* of the implied edit.

In Example 1, we used Marital Status as the generating field. The codes for this field were proper subsets of the set of all possible marital status codes in Edits e_1 and e_2 . However, in the implied edit e^* , the code set corresponding to this field is the set of all possible codes of Marital Status. Thus, e^* would be called an *essentially new implied edit*. However, using Age as the generating field, we would obtain

$$(\text{Age} = \text{Any code}) \cap (\text{Rel. to Head} = \text{Spouse}) \cap (\text{Mar. Stat.} = \text{Divorced, Widowed, Separated}) = F.$$

This would not be an *essentially new implied edit* according to our terminology, because in one of the edits (e_2) the generating field (Age) is represented by the set of all possible codes for that field. Intuitively, one can

also see why we would not call this edit an "essentially new implied edit": while it is an implied edit, it is simply a weaker form of e_2 and, thus, does not add to our understanding of the constraints imposed by the edits on the data.

Referring to the first paragraph of this section, we are now in a position to define unambiguously the concept of knowing all the relevant edits (explicit and implied) which the data are required to satisfy. The set of explicit (initially specified) edits, together with the set of all essentially new implied edits, is said to constitute a *complete set of edits* and is denoted by Ω . Clearly, any finite set of explicit edits corresponds to a complete set of edits.

Define Ω_K as that subset of Ω which involves only Fields 1, 2, $\dots$, K . Formally, Ω_K consists of those edits

$$e_r: \bigcap_{i=1}^N A_i^r,$$

for which

$$A_j^r = A_j \quad j = K + 1, K + 2, \dots, N,$$

where A_j is the complete set of code values of Field j . The following theorem holds.

Theorem 1: If a_i^0 ($i = 1, 2, \dots, K - 1$) are, respectively, some possible values for the first $K - 1$ fields, and if these values satisfy all edits in Ω_{K-1} , then there exists some value a_K^0 such that the values a_i^0 ($i = 1, 2, \dots, K$) satisfy all edits in Ω_K . The proof is presented in the Appendix. We just note here that it depends on the set of edits being complete with respect to the essentially new implied edits, but not necessarily all possible implied edits.

In terms of Example 1, if e_1 , e_2 , and e^* are a complete set of edits Ω corresponding to e_1 and e_2 , then the subset Ω_2 of this set, involving only Fields 1 and 2 (i.e., Age and Relationship to Head), consists of e^* (since e^* is the only edit which does not involve the third field, Marital Status). Ω_1 , the subset of edits involving only Field 1 (Age), is empty. Thus, with $K = 3$, Theorem 1 states that if a record has some code for each of Age and Relationship to Head which satisfies e^* , then there is at least one code for Marital Status which, together with the current codes for Age and Relationship to Head, would result in the record satisfying all edits in Ω_3 (e_1 , e_2 , and e^*).

Corollary 1: Suppose that a questionnaire has N fields, and assume that Fields 1, $\dots$, $K - 1$ have values a_i^0 ($i = 1, \dots, K - 1$) such that all edits in Ω_{K-1} are satisfied; then there exist values a_i^0 ($i = K, \dots, N$) such that the values a_i^0 ($i = 1, 2, \dots, N$) satisfy all edits. The proof follows immediately by repeated application of Theorem 1.

We are now in a position to state precisely the meaning of the intuitively stated paragraph at the beginning of this section.

Corollary 2: Suppose that a record (questionnaire) has N fields having the values a_i ($i = 1, \dots, N$). Suppose that S is a subset of these fields having the property that at least one of the values a_i ($i \in S$) appears in each failed edit, i.e., in each edit failed by the given record. Then values a_i^0 ($i \in S$) exist such that the imputed record consisting of the values a_i ($i \notin S$), together with a_i^0 ($i \in S$), satisfies all edits. The proof is given in the Appendix.

We now see the full impact of having a complete set of edits. We have only to select *any* set of fields having the property that at least one of them is involved in each failed edit and it then follows that a set of values exists for these fields which, together with the unchanged values of the other fields, will result in an imputed record satisfying all edits. If the set of fields which we select is the set containing the smallest number of fields (minimal set), then Corollary 2 states that by changing the values in these fields (but keeping values in other fields unchanged) we will be able to satisfy all the edits. Thus, if we have a complete set of edits, Corollary 2 provides an operational procedure whereby, given a questionnaire, the smallest number of fields can be identified, which, if changed, will result in all edits being satisfied by the given questionnaire.

It should be emphasized that the Lemma and Theorem 1 apply to all types of edits which can be expressed in the normal form. As indicated earlier, the normal form of edits is a natural language to users in describing edit inconsistencies for logical edits, applying to coded data. Linear arithmetic edits applicable to quantitative data *can* be expressed in the normal form, although for these latter types of edits the normal form does not represent a natural language. Thus, as will be seen in Section 5, the Lemma provides the basis for an operationally feasible method of deriving implied edits from logical edits. While, in the case of linear arithmetic edits, the Lemma does not directly result in an operational procedure of deriving implied edits, it still holds as a theoretical result, together with Theorems 1 and 2. In Section 5 we will develop an appropriate algorithm applicable to linear arithmetic edits which can be shown to be equivalent to the procedure of the Lemma.

While the Lemma provides a method of generating essentially new implied edits, it does not guarantee that repeated application of that method will, in fact, generate all essentially new implied edits. That this is, in fact, the case is stated by Theorem 2.

Theorem 2: If

$$e_p: \bigcap_{i=1}^N A_i^p = F$$

is an edit which is logically implied by the explicit edits, then it can be generated by the edit generation procedure of the Lemma. The proof is given in the Appendix.

A useful result of the method of generating implied edits is that any internal inconsistency in the explicit

edits is identified. We have an intuitive notion of what inconsistent edits may mean: requirements, concerning the data, that are self-contradictory. However, as just indicated, an edit represents a restriction on the code space, and a series of edits represents a series of such restrictions. They cannot, in themselves, be contradictory. They can, however, contradict the initial field by field identification of permissible code values. Thus, a set of edits is said to be *inconsistent* if they jointly imply that there are permissible values of a single field which would automatically cause edit failures, irrespective of the values in the other fields (clearly, such values should not be in the set of possible values for the field). Thus, an inconsistent set of edits means that there is an implied edit e_r of the form

$$e_r: A_i^r = F,$$

where A_i^r is a proper subset of the set of possible values for some Field i . However, if A_i^r is a subset of the possible values of Field i , then this edit could not be an originally specified edit. Since the edit generating process identifies all implied edits, it follows that this edit will also be generated. It is a simple matter to computer check the complete set of edits to identify implied edits of this type and, thus, to determine whether the set of originally specified edits is inconsistent.

4. IMPUTATION

We have developed a method which, corresponding to any current record, identifies a set of fields (in particular the smallest set) whose values can be changed in such a fashion that the resulting record would satisfy all edits. The next step is the choice of suitable values for those fields. We will consider a number of ways of performing this second step.

Primarily we will confine ourselves to the so-called "hot-deck" type imputation methods. These essentially consist of imputing for a field of the current record the value recorded in the same field of some other record which, however, passed all the relevant edits. This method attempts to maintain the distribution of the data as represented by the records which passed the edits.

Let us assume that we have a record $\{a_i^0: i = 1, \dots, N\}$ of which the first K fields are to be imputed, this being the minimal set of fields. We assume throughout the sequel that we have a complete set of edits.

Method 1, Sequential Imputation: Let us begin by imputing for Field K and then systematically impute for Fields $K - 1, K - 2, \dots, 1$.

Consider all of the M edits (if any) in which Field K is specifically involved, but *not* Fields $1, 2, \dots, K - 1$.

$$e_r: \bigcap_{i=K}^N A_i^r = F \quad r = 1, \dots, M. \quad (4.1)$$

Since the K th Field is explicitly involved in each edit, A_K^r is never the whole set of possible values of Field K .

Now consider a given record. Clearly, among the edits in (4.1), we may disregard those which the given record satisfies *on account of its values in Fields $K + 1, K + 2, \dots, N$* , since these edits will remain satisfied irrespective of what value we impute in Field K . In other words, for a given record we may disregard those edits in (4.1) for which

$$a_i^0 \notin A_i^r, \quad \text{for at least one } i = K + 1, K + 2, \dots, N,$$

and consider only those M' edits for which

$$a_i^0 \in A_i^r, \quad \text{for all } i = K + 1, K + 2, \dots, N; \\ r = 1, \dots, M'.$$

Loosely speaking, these are all the edits among (4.1) which the given record fails or may fail, depending on its value in Field K . Now if we want to satisfy all these edits by imputing a value to Field K , the value to be imputed must satisfy

$$a_K^* \in \bigcap_{r=1}^{M'} \bar{A}_K^r, \quad (4.2)$$

where $\bar{A}_K^r$ is the complement of A_K^r with respect to A_K . This is always possible, since, if the set on the right side of (4.2) was empty, it would mean, according to Theorem 1, that there is an edit which the given record fails and which involves only Fields $K + 1, K + 2, \dots, N$. This is contrary to the choice of the Fields $1, 2, \dots, K$ for imputation.

Having imputed a_K^* , we have all edits satisfied which involve Field K and any of the Fields $K + 1, K + 2, \dots, N$ (but which do not involve Fields $1, 2, 3, \dots, K - 1$). We next consider all edits involving Fields $K - 1, K, \dots, N$, but not Fields $1, 2, \dots, K - 2$ and will satisfy these, as before, by imputing a suitable value for Field $K - 1$ (leaving all other fields unchanged). We continue until all the Fields $1, 2, \dots, K$ have been imputed, and hence, by the construction, all the edits satisfied.

Example 3: Suppose that a record contains five fields, each with its possible set of codes.

Sex	Age	Mar. Stat.	Rel. to Head	Education
Male	0-14	Single	Wife	None
Female	15-16	Married	Husband	Elementary
	17+	Divorced	Son or Daughter	Secondary
		Separated	Other	Post-secondary
		Widowed		

The following five edits apply (they are a complete set).

- $e_1: (\text{Sex} = \text{Male}) \cap (\text{Rel. Head} = \text{Wife}) = F$
- $e_2: (\text{Age} = 0-14) \cap (\text{Mar. Stat.} = \text{Ever Married}) = F$
- $e_3: (\text{Mar. Stat.} = \text{Not Married}) \cap (\text{Rel. Head} = \text{Spouse}) = F$
- $e_4: (\text{Age} = 0-14) \cap (\text{Rel. Head} = \text{Spouse}) = F$
- $e_5: (\text{Age} = 0-16) \cap (\text{Educ.} = \text{Post-secondary}) = F$

Let the current record contain the characteristics

Field 1 = Sex = Male

Field 2 = Age = 12

Field 3 = Mar. Stat. = Married

Field 4 = Rel. Head = Wife

Field 5 = Educ. = Elementary.

It is easy to verify that Edits e_1, e_2, e_4 fail. No single field covers off all these edits. There are three pairs of fields which do: Sex and Age, Age and Relationship to Head, Marital Status and Relationship to Head. Suppose a decision has been made to impute Sex and Age, leaving the other three fields unchanged (thus $K = 2$). Now consider all edits which involve Age (the K th variable), but do not involve Sex (variable $K - 1 = 1$): these are the edits e_2, e_4, e_5 . Thus, $M = 3$. However, e_5 is satisfied on account of the values of Fields $K + 1, K + 2, K + 3$ (the three fields which are not imputed). Specifically, because Education $\neq$ Post-secondary, e_5 will be satisfied whatever Age we impute; thus, e_5 need not be considered when imputing Age (therefore, $M' = 2$). So we have to consider the sets $\bar{A}_2^2$ and $\bar{A}_2^4$ (the complements of the code sets for Age in Edits e_2 and e_4). According to (4.2), we have to choose an Age value a_2^* such that

$$a_2^* \in \bar{A}_2^2 \cap \bar{A}_2^4 = (\text{Age} = 15+) \cap (\text{Age} = 15+) \\ = (\text{Age} = 15+) .$$

Thus, we can impute any age greater than or equal to 15 years.

Having imputed Age (suppose 22 years), we next impute Sex. Again, we consider all edits involving Sex (there are no other fields to impute). Only e_1 involves Sex. We check whether e_1 is satisfied on account of the values in the fields already imputed or left unchanged—this is not the case. Thus, now $M = M' = 1$. Hence, we have to impute a code, a_1^* , for sex such that

$$a_1^* \in \bar{A}_1^1 = (\text{Sex} = \text{Not Male}) .$$

Clearly, the only feasible imputation is Sex = Female.

It should be pointed out that this method appears to be predicated on being able to find, at each step, edits which involve only one of the fields to be imputed, together with fields which do not need to be imputed or which have already been imputed.

We may, however, reach a point where we cannot choose a field satisfying this criterion. Suppose that after $K - k$ fields have been imputed according to this procedure, there is no failed edit which involves Field k without some of the remaining Fields $k - 1, k - 2, \dots, 1$. In this case we may simply impute any value for Field k and then continue with Field $k - 1$.

Conceptually, the actual imputation can be as follows: Having determined from (4.2) the set of acceptable values, one of which must be imputed, we search (with a random starting point) among the records which passed all edits or among those which were already imputed but for which the given field was *not* imputed. We accept as

the current imputed value for each field the first acceptable values (as defined by (4.2)) encountered during the search. By so doing, the various acceptable values will be chosen with probability proportional to their occurrence in the population as a whole, as long as the errors among the original records occur more or less in random order.

The procedure is subject to some obvious disadvantages. We are not taking full account of other information in the record which might be associated (correlated) with the field being imputed, except that the imputed value will be consistent with the largest possible number of originally reported fields in the record. Also, since the fields are imputed one by one, the joint distribution of values imputed to individual fields will be different from that in the population (except when the distributions involved are independent), although their marginal distribution will be the same. This last disadvantage is true of most sequential imputation methods. The method can be improved if we restrict the search to those records which have in some specified other fields identical (or similar, according to some definition) values to those in the current record.

Method 2, Joint Imputation: Consider all the edits. Assume that Fields 1 to K are to be imputed and consider, as with Method 1, only those M'' edits which the given record can potentially fail, depending on the choice of values for imputation to Fields $1 \dots K$, i.e.,

$$e_r: \bigcap_{i=1}^N A_i^r = F, \quad \text{where } a_i^0 \in A_i^r \\ \text{for all } i = K + 1, \dots, N; r = 1, \dots, M'' .$$

Consider the sets

$$A_i^* = \bigcap_{r=1}^{M''} A_i^r, \quad i = K + 1, \dots, N . \quad (4.3)$$

A_i^* cannot be empty, since $a_i^0 \in A_i^r, r = 1, \dots, M''$. If we choose any previously processed (i.e., edited and imputed) record, whose values in Fields $K + 1, \dots, N$ are within the corresponding sets (4.3), then, since that record satisfies all the edits, its values in Fields 1, 2, $\dots, K$ may be used for the current imputation and will automatically satisfy all the edits.

Note that there is now no need to calculate the possible values of the fields to be imputed, but we do have to identify the sets (4.3) of other fields. These sets identify "ranges" for each of the fields left unchanged in such a way that any other previously accepted record, whose values in Fields $K + 1$ to N fall in these ranges, will provide suitable values for imputation in Fields 1 to K .

This is not likely to be as difficult as it might first appear, since the number of edits for which $a_i^0 \in A_i^r$ for all $i = K + 1, \dots, N$ may not be too large.

We illustrate this procedure using Example 3. There are two fields to impute, Sex and Age. We first consider all the edits which these two fields enter: e_1, e_2, e_4, e_5 . Of these, e_5 will be satisfied whatever we impute for Sex

and Age because the Education code of the current record is Elementary school (i.e., not Post-secondary). Thus only e_1 , e_2 and e_4 need to be considered when imputing Age and Sex: $M'' = 3$. The sets A_i^* of (4.3) are constructed as follows.

$$\begin{aligned} A_3^* &= A_3^1 \cap A_3^2 \cap A_3^4 = (\text{Mar. Stat.} = \text{Any code}) \\ &\quad \cap (\text{Mar. Stat.} = \text{Ever Married}) \\ &\quad \cap (\text{Mar. Stat.} = \text{Any code}) \\ &= (\text{Mar. Stat.} = \text{Ever Married}) \\ A_4^* &= A_4^1 \cap A_4^2 \cap A_4^4 = (\text{Rel. Head} = \text{Wife}) \\ &\quad \cap (\text{Rel. Head} = \text{Any code}) \\ &\quad \cap (\text{Rel. Head} = \text{Spouse}) \\ &= (\text{Rel. Head} = \text{Wife}) \\ A_5^* &= A_5^1 \cap A_5^2 \cap A_5^4 = (\text{Education} = \text{Any code}). \end{aligned}$$

So if we search the previously processed "clean" records until we encounter one whose Marital Status is one of the codes subsumed under Ever Married, and whose Relationship to Head code is Wife, and whose Education code is any code, we can impute for the current record the Sex and Age codes of that record.

Note that this method may also be simply extended to take account of information in the other fields which are not explicitly involved in edits with the particular fields to be imputed, but which are thought to be closely associated. In fact, if the number of fields and the number of codes for each field is not too large, one may altogether avoid the determination of the "ranges" A_i^* and simply borrow the values to be imputed from a record which passed all edits and which, in all relevant fields not to be imputed, agrees identically with the given record. Here "relevant" implies involvement in edits which may potentially fail depending on the choice of values for imputation.

In terms of Example 3, Marital Status and Relationship to Head were relevant for the imputation of Sex and Age, but Education was not. Note that the concept of which fields are relevant in this sense may vary from one record to another. If the record in Example 3 had an Education code of Post-secondary, then Education would also have been relevant in imputing Age.

This method takes much more account of the information contained in the other data fields of the record to be imputed. The sets of values A_i^* essentially define jointly a subset of the whole population to which the record under consideration belongs and which is smaller than the whole population. The use of other fields thought to be well associated with the field to be imputed strengthens the use of the information contained within the record. On the other hand, since we are searching for a record belonging to a smaller subset of the population, it will probably take longer than with Method 1 to find a suitable record—although, at least partly offsetting this is the fact that there is only one search instead of K .

Since the imputation is done jointly for the K fields, it is not just the values of the single fields which will now appear in the same proportions as in the population, but also the incidence of combinations of values. Thus, no longer is there a danger of imputing into the same

record two reasonably common values which almost never occur together.

Clearly, the relative merits of Methods 1 and 2 depend on the complexity of the edits, the amount of information contained in some fields in a questionnaire about others, the number of fields and their possible code values, and the frequency of edit failures. Almost certainly any attempt to use Method 2 will need a default option in case no previously accepted record can be found within a reasonable search period. In fact, the system implemented at Statistics Canada uses Method 2 as the main imputation procedure, with Method 1 as a default option.

5. IMPLEMENTATION PROCEDURES

This section is devoted to an outline of a number of practical methods developed to facilitate the implementation of the previous sections. A generalized system for editing and imputation has, in fact, been developed based on the concepts of this article.²

5.1 The Application of Logical Edits to a Record

The entire implementation strategy of a system of automatic editing and imputation hinges on the possibility of representing logical edits (in the normal form) as well as coded data records in the form of extremely simple strings of 0's and 1's ("bits," according to computer terminology). The method will be illustrated using Example 3.

The column heading of Table 1 corresponds to a description of all possible codes of all possible fields of Example 3: one column corresponds to each possible code, a set of columns corresponds to one field. Each line in the table is a representation of one edit.

Each edit in the normal form can be fully characterized in terms of the code sets with which each field enters that edit. For example, e_3 can be written as

$$\begin{aligned} e_3: & (\text{Sex} = \text{Any code}) \cap (\text{Age} = \text{Any code}) \\ & \cap (\text{Mar. Stat.} = \text{Not Married}) \cap (\text{Rel. Head} = \text{Spouse}) \\ & \cap (\text{Education} = \text{Any code}) = F. \end{aligned}$$

This edit is fully characterized by the five code sets corresponding to the five fields. These code sets, in turn, can be characterized by entering in Table 1 an entry of 1 for each member of the set, and 0 elsewhere. Thus, for e_3 , we enter 1's in the columns corresponding to the two possible Sex codes, the three possible Age codes, the four possible Education codes; however, of the five possible Marital Status codes, only the four columns corresponding to the description Not Married are completed with 1; in the Married column we enter 0; and of the four columns corresponding to the Rel. Head codes we enter 1 only in the two columns of Husband and Wife (i.e., Spouse), putting 0's in the other two columns. Table 1 thus provides a unique representation of the five edits of Example 3. It is called the *logical edit matrix*.

² For further information about the system, write M. Podehl or R. Graves, General Survey Systems Staff, Statistics Canada, Ottawa, Canada.

1. Representation of the Five Edits and One "Current Record" of Example 3

Edit	Sex		Age			Marital status					Rel. to head				Education			
															None	Ele- men- tary	Sec- ond- ary	Post- Sec- ond- ary
	Male	Female	0-14	15-16	17+	Single	Mar- ried	Di- vorced	Sepa- rated	Wid- owed	Wife	Hus- band	Son, Daugh- ter	Other				
e_1	1	0	1	1	1	1	1	1	1	1	1	0	0	0	1	1	1	1
e_2	1	1	1	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
e_3	1	1	1	1	1	1	0	1	1	1	1	1	0	0	1	1	1	1
e_4	1	1	1	0	0	1	1	1	1	1	1	1	0	0	1	1	1	1
e_5	1	1	1	1	0	1	1	1	1	1	1	1	1	1	0	0	0	1
Current record	1	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	0	0

Having represented all edits in this fashion, we can similarly represent a data record. To represent the current record of Example 3, under each field we enter 1 in one of the columns, 0's elsewhere: the column where 1 is entered corresponds to the code for that field of the current record. Comparing the last row of Table 1 with the current record of Example 3, the representation becomes almost self-explanatory.

Now it is very easy to apply to our current record each of the edits. Conceptually, we superimpose the current record on each of the rows corresponding to the edits. It is easy to verify that the current record fails an edit if and only if all the 1's of the data record are overlaid on 1's in the row corresponding to that edit. Put differently, if an edit is viewed as a vector of 0's and 1's (in this example, a vector of 18 components) and if the current record is similarly viewed, the data record would fail an edit if and only if the scalar product of the two vectors is equal to the number of fields (five, in this example).

A faster and more effective equivalent algorithm would consist of selecting from the preceding edit matrix those columns corresponding to the code values of the current record. In the example, we would obtain for the current record the five first columns in Table 2. We add a last column, which is simply the product of the entries in each of the rows. 0 in the last column indicates that the current record passes the edit; 1 indicates that the edit is failed. Thus, we see once again that the record of Example 3 passes Edits e_3 and e_5 , but it fails Edits e_1 , e_2 , and e_4 . The validity of this algorithm is easily verified.

It can be readily seen that if a given edit is represented by a string of 1's for every possible code of a given field (such as Sex in e_2), this indicates that a record will pass or fail that edit irrespective of its code in the given field.

2. Representation of Edit Algorithm

Edit	Male	0-14	Mar- ried	Wife	Elemen- tary	Prod- uct
e_1	1	1	1	1	1	1
e_2	1	1	1	1	1	1
e_3	1	1	0	1	1	0
e_4	1	1	1	1	1	1
e_5	1	1	1	1	0	0

Thus, Edit e_2 will pass or fail depending on the code for Age and for Marital Status—but irrespective of the code for Sex, Relationship to Head, or Education. No edit can be represented by a set of 0's for all code values in a field, since this would imply that the edit could not be failed by any record.

Single-field edits can also be included in the representation illustrated by Table 1. All we need do is expand the code set for every field by the inclusion of an extra code labeled "Invalid." Then a single field validation edit, say, for Sex, would be represented by 1 in the column corresponding to the Invalid code of the field Sex, the other columns of Sex being filled out with 0's, and all other columns of all other fields being filled out with 1's (indicating that for this particular edit the fields other than Sex are irrelevant, and as far as Sex is concerned, the only code which would fail this edit would be a code Invalid).

Note that, as far as editing only is concerned, implied edits need not be considered—if the initially stated explicit edits all pass, no implied edit can fail. Thus, e_4 would not need to be considered, since it has been shown to be implied by e_2 and e_3 . Nevertheless, when we come to imputation, the implied edits become relevant.

5.2 The Derivation of a Complete Set of Logical Edits

Having represented logical edits in the form of the logical edit matrix, it is easy to devise a procedure implementing the Lemma for generating new edits.

To illustrate, refer to Table 1. Let us derive an implied edit from e_2 and e_3 , using Marital Status as the generating field. The construction of the new edit from e_2 and e_3 (represented as in Table 1) can be graphically illustrated.

Edit	Sex	Age	Mar. Stat.	Rel. to Head	Education
e_2	1 1 1 0 0 0	1 1 1 1 1 1	1 1 1 1 1 1	1 1 1 1 1 1	1 1 1 1 1 1
e_3	1 1 1 1 1 1	0 1 1 1 1 1	1 1 1 1 1 1	1 0 0 1 1 1	1 1 1 1 1 1
	and	and	or	and	and
Implied edit	1 1 1 0 0 1	1 1 1 1 1 1	1 1 1 1 1 1	1 1 0 0 1 1	1 1 1 1 1 1

In the representation of the implied edit for each field, except the generating field, we enter 0 for a code if any of the contributing edits has 0 there—entering 1 only if all the contributing edits have 1 in that code. For the generating field this is reversed: we enter 1 for a code if

any of the contributing edits has 1 there—entering 0 only if all the contributing edits have 0 in that code.

The new edit produced is a valid essentially new implied edit unless

1. One of the fields contains all 0's.
2. The generating field does not contain all 1's.
3. The new edit is already contained in an existing edit.

This last condition is easily checked, since, for the new edit to be redundant, there must be an edit already identified which has a 1 in every location where the new edit has a 1. Note that the new edit is already in the normal form (2.7) and can be added to the list of edits.

This "implied edit" satisfies both of the first two criteria just listed. In fact, it is Edit e_4 of Table 1. Thus, because of Condition 3, it would not be added to the list of edits (e_4 is already part of that list).

In very simple cases one could combine all edits in pairs, triples and higher combinations, using each field as the generating field in turn, until all combinations have been exhausted. It is clear that for all but the simplest of situations this would be tedious and a screening operation is required for fruitless combinations. The following rules could form the basis of a suitable algorithm. The proof of each rule has either been given previously or is straightforward.

1. For any set of edits to produce an essentially new edit, they must have a field in common which is specifically involved in each of them (through a proper subset of its values).
2. No essentially new edit will be produced from an implied edit and a subset of the edits from which it was itself generated.
3. A combination of edits using a particular Field i as the generating field need not be considered if some subset of the proposed combination using the same Field i has already resulted in an essentially new implied edit.

Remember that any new edit which involves just one field indicates that the original explicit edits are themselves inconsistent. Remember also, that having derived all implied edits, it is necessary to add the single field valid code checks to form the complete set of logical edits.

With these simple conditions it is possible to develop an algorithm to generate all essentially new implied edits and at the same time insure that the explicit edits are mutually consistent. In fact, the general system for editing and imputation, mentioned earlier, includes a module for the derivation of all essentially new implied edits. Note that the consistency of the edits is validated at the edit generation stage before data processing begins, when corrections are easily made, rather than at the data processing stage when, through a chance combination of data values, the belated discovery of the existence of inconsistencies could cause extensive dislocation of processing schedules, together, possibly, with the need to reformulate the edits and reprocess the data.

5.3 The Derivation of a Complete Set of Arithmetic Edits

It is easy to derive formal rules which correspond, in the case of arithmetic edits involving linear expressions,

to a repeated application of the Lemma to generate essentially new implied edits. To save space, we consider only the case where the fields clearly divide into two classes: those involved in only logical edits and those involved in only arithmetic edits. In such a case, a complete set of edits can separately be derived for the logical edits (using the method of Section 5.1) and for the arithmetic edits (as outlined shortly). We also restrict ourselves to linear arithmetic edits.

A linear edit, expressing an edit failure, takes the form

$$f(a_1, \dots, a_N) \geq 0,$$

where the inequality will be strict according to the original edit specification, and where f is a linear function of the variables $a_1, a_2, \dots, a_N$. Any record for which $f \geq 0$ fails this particular edit. It is easy to see that as well as edit failures specifically of this type, this form also includes the two most common other types of arithmetic edits: where a linear identity must be satisfied and where a linear expression must be within a constant range (both of which can be translated into two inequalities representing edit failures). Similarly, a rational expression of the fields within a constant range can also be translated into edits of this form. Clearly, by multiplying by plus or minus one the inequality can always be standardized in the direction just shown.

Since we have limited ourselves to linear edits, each expression is completely described by $N + 1$ coefficients and an indicator of the type of inequality (strict or otherwise).

<i>Edit</i>	<i>Constant</i>	<i>Field 1</i>	<i>Field 2</i>	<i>Field N</i>	<i>Indicator</i>
e_r	α_0^r	α_1^r	α_2^r	α_N^r	δ^r

For any Field i not involved in the edit e_r , α_i^r is zero. For purposes of standardization the first nonzero coefficient should be ± 1 (this can always be achieved by dividing through by its absolute value). $\delta^r = 1$ in the case of strict inequality, zero in the case of weak inequality. The derivation of a complete set of arithmetic edits is now achieved as follows.

Theorem 3: An essentially new implied edit e_t is generated from edits e_r and e_s using Field i as a generating field if and only if α_i^r and α_i^s are both nonzero and of opposite sign. The coefficients of the new edit, α_k^t , are given by

$$\alpha_k^t = \alpha_k^s \alpha_i^r - \alpha_k^r \alpha_i^s; \quad k = 0, 1, \dots, N$$

where r and s are so chosen that $\alpha_i^r > 0$ and $\alpha_i^s < 0$ and $\delta^t = \delta^r \delta^s$. These coefficients of the new edit need to be standardized by making the first nonzero coefficient equal to ± 1 .

In effect, the theorem simply states that from two linear inequalities where the inequality signs are in the same direction, a variable can be eliminated by taking their linear combinations if and only if the variable has coefficients in the two inequalities which are of the oppo-

site sign. The proof is contained in the appendix. It can also be shown that, in the case of arithmetic edits, repeated application of Theorem 3 will derive all essentially new implied edits. The proof follows directly from that of Theorem 2, but it is rather cumbersome and will be omitted.

Thus, in this most common of situations, when simple inequalities express edit failures, obtaining a complete set of edits is quite straightforward. By simply combining the suitable edits using as generating fields all possible fields, a complete set of edits will be achieved.

Note that the set of coefficients of the arithmetic edits e_r can be used in a fashion analogous to the method of Section 5.1 to determine which edits are failed by a given record. As in Section 5.1, we start with the initial explicit edits; if the record passes all of these, the implied edits need not be considered. As before, every record which has an invalid code in a field is considered as failing all edits which that field enters explicitly.

5.4 Identification of the Minimal Set of Fields for Imputation

Whether the edits be logical, arithmetic or a mixture, once a complete set has been developed we are in a position to process the records. If any edits are failed by a particular record, then the minimal set of fields for imputation needs to be identified.

Consider an $(R' \times N)$ matrix, R' being the number of failed edits in the complete set (including the single field edits) and N the number of fields on the questionnaire. Let the (r, n) th cell of the matrix be zero unless field n is specifically involved in edit r (the edit being failed by the data values of the record under consideration).

For example, the following matrix indicates that Edit 1 is satisfied and, thus, does not appear in the matrix, Edit 2 is failed and Fields 1, 2 are involved in it, etc., Edit R is failed and Fields 2, 3, . . . , N are involved in it.

Edit	Field							
	1	2	3	.	.	.	.	N
e_2	1	1	0	-	-	-	-	0
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
e_R	0	1	1	-	-	-	-	1

This matrix, which we term the failed edit matrix, can be generated by computer.

The problem of identifying the smallest number of fields to be changed to make all edits satisfied is now reduced to the problem of choosing the smallest set of fields (columns) which together have at least 1 in each failed edit (row).

Clearly, in simple cases we could examine each field (column) separately to see if it entered all the failed edits (rows), and then each pair of fields, and then triples, and so on. If we expect most records to be corrected by the imputation of very few fields, this may be satis-

factory, but in the case of a large questionnaire this could be very slow.

A possible alternative procedure is as follows.

- 1. All satisfied edits may be ignored (rows containing zeros only).
- 2. Identify all fields failing single field edits (validity checks, incorrect blanks, etc.). By definition these must be included in the minimal set. All failed edits which explicitly involve these fields will now be covered off, so we can generate a modified failed edit matrix by deleting all edits in which the chosen fields appear explicitly. If no edits remain, the minimal set has been identified.
- 3. If Step 2 did not eliminate all edits, identify the edit involving the fewest fields (select an edit arbitrarily if there is a tie). At least one of the fields involved in this edit must be in the minimal set.
- 4. For each such field generate a modified failed edit matrix as in Step 2. Keep a record of the combination of fields so far selected.
- 5. Repeat Steps 3 and 4 until the first modified failed edit matrix vanishes.

To illustrate the algorithm, we return to Example 3. Suppose a current record (different from Table 1) is as follows.

Sex = Male
Age = 0-14
Mar. Stat. = Divorced
Rel. to Head = Wife
Education = Post-secondary

A look at the edit representation part of Table 1 will show that all five edits are failed, so we get the following failed edit matrix for the given current record.

Edits	Sex	Age	Mar. Stat.	Rel. to Head	Education
e_1	1	0	0	1	0
e_2	0	1	1	0	0
e_3	0	0	1	1	0
e_4	0	1	0	1	0
e_5	0	1	0	0	1

A 1 corresponding to, say, Sex and Rel. to Head in e_1 indicates that these fields explicitly enter e_1 ; similarly, the other edits. The structure of this tabulation is immediately derived from Table 1 and the current record.

Now we start a cycle as just indicated.

Cycle 1: Select an edit involving the fewest number of fields. In the present case all edits involve two fields, so we arbitrarily select e_1 . Fields in minimal set after Cycle 1: Sex, or Rel. to Head.

Next, eliminate all the edits which involve either Sex or Rel. to Head. We obtain two modified failed edit matrices, corresponding to the choice of Sex or Rel. to Head for imputation.

Cycle 2: The following tabulation shows the modified failed edit matrix if Sex is to be imputed from Cycle 1.

Edits	Age	Mar. Stat.	Rel. to Head	Education
e_2	1	1	0	0
e_3	0	1	1	0
e_4	1	0	1	0
e_5	1	0	0	1

Selecting e_2 (arbitrarily, since all edits involve the same number of fields), we could impute Age or Mar. Stat.

Possible fields in minimal set:

Sex and Age

Sex and Mar. Stat.

If we select Rel. to Head in Cycle 1, we obtain the modified failed edit matrix shown in the following tabulation.

<u>Edits</u>	<u>Sex</u>	<u>Age</u>	<u>Mar. Stat.</u>	<u>Education</u>
e_2	0	1	1	0
e_5	0	1	0	1

Selecting e_5 arbitrarily, we would get the following possible fields in minimal set:

Rel. to Head and Age

Rel. to Head and Education.

Corresponding to the four possible sets of fields in the minimal set after Cycle 2, we get the following four modified failed edit matrices.

If Sex and Age is imputed from Cycle 2, we obtain

<u>Edit</u>	<u>Mar. Stat.</u>	<u>Rel. to Head</u>	<u>Education</u>
e_3	1	1	0

If Sex and Mar. Stat. is imputed from Cycle 2, we obtain

<u>Edit</u>	<u>Age</u>	<u>Rel. to Head</u>	<u>Education</u>
e_4	1	1	0
e_5	1	0	1

If Rel. to Head and Age is imputed from Cycle 2, we obtain

Empty .

Finally, if Rel. to Head and Education is imputed from Cycle 2, the modified failed edit matrix becomes

<u>Edit</u>	<u>Sex</u>	<u>Age</u>	<u>Mar. Stat.</u>
e_2	0	1	1

Since, at the end of Cycle 2 we obtained an empty modified failed edit matrix, we do not need to continue: the fields in the minimal set are Rel. to Head and Age.

Of course, in this simple example the result could be directly verified from the first failed edit matrix: Rel. to Head and Age between them cover off all failed edits, but no other pair of fields does.

Other approaches may be developed to the problem which could well be more efficient. The proposed procedure is a relatively simple iterative one for identifying all possible minimal sets for imputation purposes.

5.5 Imputation

We will now translate the two methods of imputation proposed in Section 4 into the representation developed in this section and suggest algorithms for both methods. This will be done in terms of Example 3 of Section 4.

The same example is represented by Table 1 of Section 5.1. Thus, both the edits and current record are as shown in Table 1 (the current record of Section 5.4 was different!).

As in Section 4, we have decided to impute for the current record the fields Sex and Age. This could have been an outcome of the algorithm outlined in Section 5.4. (For purposes of illustration, another current record was used.)

Method 1: Suppose we impute first Age (Field 2), then Sex (Field 1).

1. Identify the edits which are satisfied on account of the values which the current record has in the other three fields, irrespective of what we may impute for Age and Sex. A look at the column of Table 1 corresponding to the current value of Mar. Stat. (= Married) indicates that Edit e_3 will be satisfied irrespective of the current values of any of the other fields. The current value of Rel. to Head (= Wife) does not result in the automatic satisfaction of any edit, but the current value of Education (= Elementary) results in e_5 being satisfied irrespective of the other fields.

Thus, only e_1 , e_2 , and e_4 can possibly impose constraints on the imputation of Age and Sex.

2. Consider those edits among e_1 , e_2 , and e_4 which involve Age but *not* Sex. Again, a look at Table 1 shows that e_1 does not involve Age. The edits e_2 and e_4 both involve Age and neither involves Sex. So we have to impute for Age in such a way that these two edits are satisfied. Theorem 1 insures that this can always be done. Looking at the Age portion of e_2 and e_4 , we get from Table 1

<u>Edit</u>	<u>0-14</u>	<u>15-16</u>	<u>17+</u>
e_2	1	0	0
e_4	1	0	0

If we combine column by column the codes in such a way that the resulting row contains 1 whenever any of the rows contained 1, otherwise it contains 0, we obtain in this simple case the representation of possible imputations for Age.

<u>0-14</u>	<u>15-16</u>	<u>17+</u>
1	0	0

Now if we impute for Age any code where the preceding representation contains 0, we will have both Edits e_2 and e_4 satisfied. So the acceptable imputations for Age are 15-16 or 17+.

3. We search (with a random starting point) the previously accepted records until we find one with one of the Age codes just listed. We impute the value of Age from that record.

4. In this simple example we now only have Sex to impute (more generally, we would proceed sequentially field by field repeating Steps 1, 2, and 3). The edits among e_1 , e_2 , and e_4 which involve Sex are: e_1 alone. The

only possible value of Sex which results in e_1 being satisfied is Female (the only 0 entry for Sex in e_1). So the acceptable imputation for Sex is Female.

5. Repeat Step 3 for Sex.

Method 2:

1. This step is identical with that of Method 1. We need to concern ourselves only with e_1 , e_2 , and e_4 , since these are the only edits which may possibly fail, depending on the imputed values for Age and Sex—thus, it is these we have to be careful to satisfy.

2. Consider the fields *not* to be imputed: Mar. Stat., Rel. to Head, Education. We want to consider the possible constraining effect of the current codes of these fields on the imputations which are to be made. It is easy to verify, on the basis of the material in Section 5.1, that this procedure will result in an imputation which will pass all the edits.

For each field *not* to be imputed, identify those codes (columns of Table 1) which contain 1 in each of the relevant edits: e_1 , e_2 , e_4 . This corresponds to the logical AND or set intersection operation for these code sets.

We obtain

Mar. Stat.:	Married, Divorced, Separated, Widowed
Rel. to Head:	Wife
Education:	Any code.

3. Search (with a random starting point) the previously accepted records until one is found which in the fields *not* to be imputed has any combination of the codes just listed. Impute from that record its values in those fields for which the current record must be imputed. For example, if the first such record found has the following values in the fields not to be imputed: Mar. Stat. = Separated, Rel. to Head = Wife, Education = Secondary; and if for that record Age = 22, Sex = Female, then we impute for the current record Age = 22, Sex = Female.

It is a simple matter to modify these methods to utilize other information in the questionnaire which is not linked explicitly through the edits to the fields requiring imputation. The problem is simply one of limiting the search of previously processed records to those that have particular values in particular fields, determining the acceptable values or ranges of values from the record requiring imputation. Thus, for example, one might as an additional constraint only impute income from records which have the same sex, age and occupation, even though there may be no explicit edit linking income with these fields.

To illustrate this point in terms of the preceding example, the imputation for Age and Sex of the particular current record was not actually constrained by Education, in the sense that the record from which these two fields would be imputed could have any code in its Education field. However, we could, as an additional constraint, insist that whenever Age is imputed but Education is not,

the record from which we impute should have the same Education status as that of the current record. As an extreme, we could insist that the record from which we impute should have identical codes to those of the current record. Of course, the more constraints we put on the record from which we impute (in addition to the constraints imposed by the logic of the edits), the longer, generally, the search will be through the previously processed records until we find a suitable one from which to impute.

6. THE BENEFITS OF THE PROPOSED METHOD

In summary, it may be useful to reiterate the advantages of the approach proposed here. These can be grouped under three different headings: methodological, systems, and subject matter benefits.

6.1 Methodological Benefits

1. The approach provides an orderly framework and philosophy for the development of edit and imputation procedures for surveys, in the sense that all procedures follow consistently and, largely, predictably from the edit specifications provided by subject-matter experts.
2. The procedure preserves the maximum amount of reported data, in that it minimizes the amount of imputation.
3. The procedure guarantees that records which have already been corrected will satisfy all the edit constraints.
4. Imputations are based on full information, i.e., the imputation of any field of a current record takes advantage of all the reported information of that record which is not subject to imputation (and which is logically related, through the edits, to the fields to be imputed).
5. For each questionnaire a log can be kept of all edits failed. This facilitates subsequent evaluation and the tracing of specific edit and imputation actions.
6. For each edit statement a log can be kept of all questionnaires failing it.
7. The uncorrected data can be preserved and cross-tabulated with the corrected data.

The major significance of these advantages lies in their evaluation possibilities. For example, a sample of questionnaires can be edited and imputed before the processing of the bulk of the survey data begins. If it turns out that some particular edits are failed by an inordinately large proportion of records, one would become suspicious of either the validity of the particular edit, the field procedures followed, or the ability of respondents to answer some particular questions. Similarly, cross-tabulation of uncorrected data with corrected data can provide revealing early feedback on the likely gross and net effect of imputations.

6.2 Systems-Oriented Benefits

1. Systems development need not be constrained by potential delays in obtaining particular subject-matter edit specifications. A generalized edit and imputation system can be developed which is independent of any particular subject-matter application. As mentioned earlier, such a generalized system has, in fact, been developed at Statistics Canada. Specific subject-matter edit specifications become input to an already implemented system.

2. The edit and imputation problem is completely defined (at least in terms of system specifications) and is therefore relatively straightforward to implement.
3. The approach lends itself, throughout, to modularity of systems. It separates clearly the following distinct stages: analysis of edits (in the form of derivation of implied edits); editing (this itself is modular, due to the nature of the normal form of edits); and, finally, imputation. Such a modular approach to systems implementation facilitates changes in specifications.

6.3 Subject-Matter-Oriented Benefits

1. Given the availability of a generalized edit and imputation system, subject-matter experts can readily implement a variety of experimental edit specifications whose impact can therefore be evaluated without extra effort involving systems development. This is particularly important given the generally heuristic nature of edit specifications.
2. Specifications need not be presented in the form of integrated flowcharts or decision tables, but in that of completely independent statements of the variety of conditions which should be considered as edit failures. Thus, the development of edit specifications can proceed simultaneously by a variety of subject-matter experts. This is an important consideration in the case of multisubject surveys, such as a census.
3. Only the edits have to be specified in advance, since the imputations are derived from the edits themselves for each current record. This represents a major simplification for subject-matter experts in terms of the workload of specifying a complete edit and imputation system.
4. Feedback is available throughout the application of the system. The first feedback, in fact, can take place prior to the availability of any data. When edits are specified by subject-matter experts, these are first analyzed to derive all essentially new implied edits. The implied edits are available for review. They have been found to be very useful in practice. Clearly, whenever an implied edit is judged inappropriate, at least one of the originally specified edits must also be inappropriate. Since it is easy to identify those initially specified edits from which a given implied edit has been derived (this is, in fact, part of the feedback from the system), such a review represents a potentially useful device to screen the originally specified edits. Other feedback features of the system include a variety of optional and default tabulations showing the number of records failing edits, the number of times each edit has been failed, etc.
5. Since each edit statement (in the normal form) is completely self-contained, and because the imputations are automatically derived from the edits, the total complex of edit and imputation system can be respecified with complete ease by dropping some edit statements previously specified or by adding new edit statements to those previously specified. This feature, together with the possibility of early feedback throughout the operation of editing and imputation, represents a powerful combination, if used with appropriate care.

7. FURTHER CONSIDERATIONS

Nonuniqueness of the smallest number of fields to be imputed: There is no reason that the method of obtaining the smallest set of fields, described in Section 3, should lead to a unique solution. In fact, it is quite possible that more than one minimal set of fields can be identified.

One method of dealing with this problem, particularly if Method 2 is used for imputation, is to identify all the minimal sets of fields which could be imputed and to accept that set which can be imputed soonest in the

search. For the cost of this more complicated computer search technique, one will likely get the benefit of finding a suitable record after a shorter search than when looking for a single set of fields to impute. Of course, one could much more simply arbitrarily or randomly decide between the alternatives.

A priori weighting of fields for reliability: It often happens that one has an *a priori* belief that some fields are less likely to be in error than others. If one field is the product of a complicated coding operation and the other is a self-coded response, then, intuitively, one is inclined to believe that the simple self-coded response is more likely to be error-free. By and large we have not tried to quantify this, believing that the data alone, by identifying the smallest possible set of fields to be changed, would more readily indicate the fields most likely to be in error. However, where there is more than one minimal set of fields, an *a priori* weight of the reliability of each field could easily be used to select among the minimal sets of fields the one to be imputed. For example, we could simply take the set with the lowest product of *a priori* weights.

Note that one could carry further the notion of *a priori* weights to the extent of choosing not the smallest number of fields which would convert all failed edits to satisfied ones, but rather the set with the smallest product of weights. The method of selecting the minimal set (see Section 5) can easily be modified to accommodate this change.

Instead of using *a priori* weights, one can edit the records in one pass through the computer and determine for each field the proportion of edits entered by the fields which are failed. This may provide a less subjective measure of the relative reliability of different fields.

APPENDIX

Proof of Lemma

If the sets A_j^* are not empty, then (3.1) is formally a valid edit. In fact, it is easy to verify that any set of values on the left side of (3.1) is included in the left side of one of the edits e_r and is, thus, an edit failure according to the edit e_r . This completes the proof.

Proof of Theorem 1

Assume that the theorem is false. Then there are values for the first $K - 1$ fields, say, a_i^0 ($i = 1, 2, \dots, K - 1$), which satisfy all the edits in Ω_{K-1} but which have the property that, for every possible value of Field K , one of the edits in Ω_K is violated.

Identify one such failed edit in Ω_K corresponding to each possible value a_K of Field K ,

$$e_r: \bigcap_{i=1}^K A_i^r = F \quad (r = 1, 2, \dots, R), \quad (A.1)$$

where

$$\begin{aligned} a_i^0 &\in A_i^r \quad (i = 1, \dots, K - 1) \\ a_K &\in A_K^r. \end{aligned}$$

Note that some proper subset A_K of the complete set of values A_K of Field K must enter each edit in (A.1), i.e., A_K^r cannot be equal to A_K since, if this were so, there would be an edit in Ω_{K-1} which is failed by $(a_1^0, a_2^0, \dots, a_{K-1}^0)$ —contradicting our original assumption.

Consider the following edit, implied by the edits (A.1), and formed using the Lemma from the preceding edits e_r , with Field K as the generating field.

$$\bigcap_{i=1}^{K-1} \left\{ \bigcap_{r=1}^R A_i^r \right\} \cap \left\{ \bigcup_{r=1}^R A_{K^r} \right\} = F. \quad (\text{A.2})$$

Since $a_i^0 \in A_i^r$ ($i = 1, 2, \dots, K-1$) for all r , the intersections $\bigcap_{i=1}^{K-1} A_i^r$ are not empty. Hence, (A.2) is the expression for a valid edit. Also, according to our assumption, every possible value a_K of Field K is included in one of the sets A_{K^r} , so

$$\bigcup_{r=1}^R A_{K^r} = A_K.$$

Therefore, (A.2) reduces to the form

$$\bigcap_{i=1}^{K-1} \left[\bigcap_{r=1}^R A_i^r \right] = F. \quad (\text{A.3})$$

Since this edit is in Ω_{K-1} and it rules out $(a_1^0, a_2^0, \dots, a_{K-1}^0)$ as an invalid combination, we have a contradiction with the way the values a_i^0 were chosen. It must follow, therefore, that the assumption was false and that, in fact, there is at least one value of Field K , say, a_K^0 , which, together with a_i^0 ($i = 1, \dots, K-1$), satisfies all edits in Ω_K .

Clearly, the indexing of the fields was arbitrary and Theorem 1 holds for any set of K fields.

Note that we implicitly assumed that (A.3), an implied edit, is a member of the set of edits. Note also that (A.3) is an essentially new implied edit according to the definition following the Lemma. Thus, the validity of the theorem depends on the set of edits being complete with respect to essentially new implied edits, but not necessarily all possible edits.

Proof of Corollary 2

Suppose, without loss of generality, that S consists of the Fields $K, K+1, \dots, N$. Thus, at least one of Fields $K, \dots, N$ appears in each failed edit. Then there are no failed edits involving only the Fields $1, \dots, K-1$, so that the values a_i ($i = 1, 2, \dots, K-1$) satisfy all edits in Ω_{K-1} . Thus, from Corollary 1, there exist values a_i^0 ($i = K, \dots, N$) such that a_i ($i = 1, \dots, K-1$), together with a_i^0 ($i = K, \dots, N$), satisfy all the edits.

Proof of Theorem 2

The statement that the edit $\bigcap_{i=1}^N A_i^p = F$ is logically implied by the explicit edits means that every record which fails this edit will also fail at least one of the explicit edits. Consider some values

$$a_i^0 \in A_i^p \quad i = 1, \dots, N. \quad (\text{A.4})$$

(This is possible, since the sets A_i^p are not empty.)

It follows from (A.4) that the record having the values a_i^0 ($i = 1, \dots, N$) must fail at least one of the explicit edits. Select one explicit failed edit corresponding to each possible value of $a_N \in A_N^p$. Suppose there are R_N such edits corresponding to all the possible values of $a_N \in A_N^p$

$$e_{r_N}: \bigcap_{i=1}^N A_i^{r_N} = F; \quad r_N = 1, \dots, R_N.$$

Now consider the following edit e^* implied by the edits e_{r_N} (using the Lemma), with Field N as the generating field.

$$e^*: \bigcap_{i=1}^N A_i^q = F,$$

where

$$A_i^q = \bigcap_{r_N=1}^{R_N} A_i^{r_N}, \quad i = 1, \dots, N-1,$$

and

$$A_N^q = \bigcup_{r_N=1}^{R_N} A_N^{r_N}.$$

This is an implied edit, since none of the intersections is empty (A_i^q includes at least the value a_i^0 ; $i = 1, 2, \dots, N-1$).

Now A_N^q includes the set A_N^p , since the edits were so constructed as to insure that every $a_N \in A_N^p$ was included in one of the R_N edits and, thus, in one of the sets $A_N^{r_N}$.

Since the value a_{N-1}^0 was arbitrarily chosen from the set of values A_{N-1}^p , an edit like e^* could be derived for every value of $a_{N-1} \in A_{N-1}^p$. Consider one such edit generated for each value of $a_{N-1} \in A_{N-1}^p$,

$$e_{r_{N-1}}^*: \bigcap_{i=1}^N A_i^{r_{N-1}} = F; \quad r_{N-1} = 1, \dots, R_{N-1}, \quad (\text{A.5})$$

where, according to the construction of the edits e^* , we have

$$A_N^{r_{N-1}} \supset A_N^p; \quad r_{N-1} = 1, \dots, R_{N-1}, \quad (\text{A.6})$$

i.e., the right side is a subset of the left side. Consider the edit implied by the edits of (A.5), using Field $N-1$ as the generating field.

$$e^{**}: \bigcap_{i=1}^N A_i^s,$$

where

$$A_i^s = \bigcap_{r_{N-1}=1}^{R_{N-1}} A_i^{r_{N-1}}; \quad i = 1, 2, \dots, N-2, N;$$

$$A_{N-1}^s = \bigcup_{r_{N-1}=1}^{R_{N-1}} A_{N-1}^{r_{N-1}}.$$

Due to the construction of this edit we have

$$A_{N-1}^s \supset A_{N-1}^p,$$

and due to (A.6) we have

$$A_N^s \supset A_N^p.$$

Continuing in this way with $N-2, N-3, \dots, 1$, we can generate an edit

$$\bigcap_{j=1}^N A_j^x = F,$$

where $A_j^x \supset A_j^p$, $j = 1, \dots, N$. Clearly, the preceding edit is failed by every record which fails the edit in the statement of Theorem 2. Hence, the edit of Theorem 2 can indeed be generated from the explicit edits by the procedure of the Lemma.

Proof of Theorem 3

To prove Theorem 3, write Edits e_r and e_s as

$$e_r: f(a_1, a_2, \dots, a_N) \geq 0$$

$$e_s: g(a_1, a_2, \dots, a_N) \geq 0.$$

A record which satisfies the first (second) inequality fails the edit e_r (e_s).

We can express the variable corresponding to the generating field which, without loss of generality, we assume to be Field N . Observing that $\alpha_N^r > 0$ and $\alpha_N^s < 0$, we obtain

$$a_N \geq f'(a_1, a_2, \dots, a_{N-1})$$

$$a_N \leq g'(a_1, a_2, \dots, a_{N-1}),$$

where the coefficients β_i^r and β_i^s of f' and g' are given by

$$\beta_k^r = \alpha_k^r / \alpha_N^r$$

$$\beta_k^s = \alpha_k^s / \alpha_N^s.$$

Now suppose that a given record has values $a_1^0, a_2^0, \dots, a_{N-1}^0$ such that

$$g'(a_1^0, a_2^0, \dots, a_{N-1}^0) \geq f'(a_1^0, a_2^0, \dots, a_{N-1}^0) . \quad (\text{A.7})$$

In this case, whatever the value of the record in the N th Field, it will fail at least one of the edits e_r or e_s . Indeed, in this case we will either have $a_N \geq g'$ or $a_N \leq f'$ or $g' > a_N > f'$. In any of these cases at least one e_r or e_s will fail. For example, if $a_N \geq g'$, then, because of (A.6), we also have $a_N \geq f'$, so e_r fails.

Thus, (A.7) is an implied edit (actually an essentially new implied edit) which can be rewritten as

$$h(a_1, a_2, \dots, a_{N-1}) \geq 0 .$$

The coefficients α_k of h are clearly given by

$$\alpha_k = \alpha_k^s / \alpha_N^s - \alpha_k^r / \alpha_N^r . \quad (\text{A.8})$$

Multiplying (A.8) by $\alpha_N^s \alpha_N^r$, we obtain Theorem 3. It is easy to verify that the strict inequality applies only when both e_r and e_s involve strict inequalities.

It remains to show that if α_N^r and α_N^s are of the same sign (but both different from zero), then no essentially new edit can be implied by e_r and e_s using this procedure. Indeed, suppose that α_N^r and α_N^s are of the same sign (say, both positive). We obtain, instead of f' and g' ,

$$\begin{aligned} e_r: a_N &\geq f'(a_1, a_2, \dots, a_{N-1}) \\ e_s: a_N &\geq g'(a_1, a_2, \dots, a_{N-1}) . \end{aligned}$$

Suppose that

$$e_t: h(a_1, a_2, \dots, a_{N-1}) \geq 0$$

is an essentially new edit implied by e_r and e_s . Select some values $a_1^0, a_2^0, \dots, a_{N-1}^0$ which fail e_t , i.e., for which

$$h(a_1^0, a_2^0, \dots, a_{N-1}^0) \geq 0 .$$

If e_r and e_s are different edits, these values can also be so chosen that

the corresponding values of f' and g' are not equal, say $f' > g'$. Now choose a value a_N^0 so that

$$f' > g' > a_N^0 .$$

Clearly, the record $a_1^0, a_2^0, \dots, a_{N-1}^0, a_N^0$ will fail e_t , yet it will satisfy both e_r and e_s . So e_t cannot be an edit implied by e_r and e_s (using Field N as the generating field) if a_N^r and a_N^s are of the same sign.

[Received May 1973. Revised June 1975.]

REFERENCES

- [1] Freund, R.J. and Hartley, H.O., "A Procedure for Automatic Data Editing," *Journal of the American Statistical Association*, 62 (June 1967), 341-52.
- [2] Naus, J.I., Johnson, T.G. and Montalvo, R., "A Probabilistic Model for Identifying Errors in Data Editing," *Journal of the American Statistical Association*, 67 (December 1972), 943-50.
- [3] Nordbotten, S., "The Efficiency of Automatic Detection and Correction of Errors in Individual Observations as Compared with Other Means for Improving the Quality of Statistics," *Bulletin of the International Statistical Institute*, Proceedings of 35th Session, Belgrade, 41 (September 1965), 417-41.
- [4] Pritzker, L., Ogus, J. and Hansen, M.H., "Computer Editing Methods—Some Applications and Results," *Bulletin of the International Statistical Institute*, Proceedings of 35th Session, Belgrade, 41 (September 1965), 442-65.
- [5] Szameitat, K. and Zindler, H.J., "The Reduction of Errors in Statistics by Automatic Corrections," *Bulletin of the International Statistical Institute*, Proceedings of 35th Session, Belgrade, 41 (September 1965), 395-417.
- [6] Yates, F., "The Use of Computers for Statistical Analysis: A Review of Aims and Achievements," *Bulletin of the International Statistical Institute*, Proceedings of 38th Session, Washington, 44 (August 1971), 39-53.